TY B. Tech (ECE AI-ML)

Semester: VI **Subject: Deep Neural Networks**

Name: **Class: TY ECE AI-ML**

Roll No: **Batch:**

Experiment No: 10

Name of the Experiment: Transfer Learning Models for classification problems and exploring Hugging face API

Teacher's Signature with date

Marks

Performed on:

Submitted on:

Aim: To fine-tune a DistilBERT model on the AG News dataset for classifying news articles into four categories: World, Sports, Business, and Sci/Tech.

Pre-requisite:

- Basic understanding of Python programming language
- Familiarity with basic concepts of machine learning

Objective:

- To preprocess and prepare the AG News dataset for training and evaluation.
- To fine-tune the DistilBERT model for multi-class news article classification.
- To evaluate the performance of the trained model in accurately classifying articles into World, Sports, Business, and Sci/Tech categories.

Components and Equipment Required:
PC with Windows 7 or onwards and Anaconda distribution with Python 3.7.

Rajat Bhati
1032232906

Experiment 10: Transfer Learning for Text Classification using Hugging Face

Aim: Fine-tune a DistilBERT model on the AG News dataset for classifying news articles into four categories: World, Sports, Business, and Sci/Tech.

Model: DistilBERT (distilbert-base-uncased) - a smaller, faster version of BERT with 66M parameters

Dataset: AG News - 120,000 training articles and 7,600 test articles across 4 balanced classes

Prerequisites: Runtime > Change runtime type > T4 GPU

Step 1: Install Required Libraries

```
!pip install transformers datasets evaluate accelerate -q
```

Step 2: Import Libraries

```
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding,
    pipeline
)
import evaluate
import numpy as np
```

Step 3: Load the AG News Dataset

AG News contains 120,000 training and 7,600 test news articles.

Labels: 0 = World, 1 = Sports, 2 = Business, 3 = Sci/Tech (30,000 articles per class)

```
dataset = load_dataset("ag_news")
print(dataset)
print("\nSample article:")
print(dataset["train"][0])
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 120000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 7600
    })
})

Sample article:
{'text': "Wall St. Bears Claw Back Into the Black (Reuters) Reuters
```

Step 4: Load Tokenizer and Tokenize the Dataset

The tokenizer converts raw text into numerical token IDs that DistilBERT can process.

truncation=True ensures articles longer than 512 tokens are cut to fit the model's max input length.

```
# Load the DistilBERT tokenizer
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased"

# See how tokenization works on a sample sentence
sample = tokenizer("NASA launches Mars rover mission")
print("Tokenized sample:", sample)

# Define the tokenization function
def tokenize_function(examples):
    return tokenizer(examples["text"], truncation=True)

# Apply tokenization to the entire dataset
tokenized_dataset = dataset.map(tokenize_function, batched=True)
print("\nTokenization complete.")
```

```
Tokenized sample: {'input_ids': [101, 9274, 18989, 7733, 13631, 3260

Tokenization complete.
```

Step 5: Load DistilBERT for Sequence Classification

We load the pre-trained DistilBERT model and add a classification head with 4 output neurons.

The warning about uninitialized weights is expected. It means the classification head is new and will be trained.

```
# Define label-to-name mapping
id2label = {0: "World", 1: "Sports", 2: "Business", 3: "Sci/Tech"}
label2id = {"World": 0, "Sports": 1, "Business": 2, "Sci/Tech": 3}

# Load model with 4-class classification head
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=4,
    id2label=id2label,
    label2id=label2id
)

print(f"Model loaded. Total parameters: {model.num_parameters():,}"
```

```
Loading weights: 100%  100/100 [00:00<00:00, 322.94it/

                       s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm
DistilBertForSequenceClassification LOAD REPORT from: distilbert-bas
Key                      | Status     |
-------------------------+------------+-
vocab_layer_norm.bias    | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_transform.weight   | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
pre_classifier.weight    | MISSING    |
pre_classifier.bias      | MISSING    |
classifier.weight        | MISSING    |
classifier.bias          | MISSING    |

Notes:
- UNEXPECTED  :can be ignored when loading from different task/archi
- MISSING :those params were newly initialized because missing from
```

Step 6: Define the Evaluation Metric

We use accuracy to measure model performance: what percentage of predictions are correct.

np.argmax converts raw prediction scores into predicted class labels.

```
accuracy = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=-1)
    return accuracy.compute(predictions=predictions, references=lab
```

Step 7: Configure Training Arguments

- **learning_rate=2e-5**: Small rate to preserve DistilBERT's pre-trained

knowledge

- **batch_size=16**: Number of articles processed per training step
- **epochs=2**: Two full passes through the 120,000 training articles
- **fp16=True**: Half-precision training for faster speed and lower memory usage

```
training_args = TrainingArguments(
    output_dir="ag-news-classifier",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    eval_strategy="steps",
    eval_steps=1000,
    save_strategy="steps",
    save_steps=1000,
    load_best_model_at_end=True,
    fp16=True,
    report_to="none",
)
```

Step 8: Initialize the Trainer and Start Training

DataCollatorWithPadding dynamically pads articles to equal length within each batch.

The Trainer handles the full training loop: forward pass, loss, backpropagation, weight updates, evaluation.

```
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

# Start training
trainer.train()
```

[15000/15000 20:43, Epoch 2/2]

Step	Training Loss	Validation Loss	Accuracy
1000	0.194456	0.241032	0.921184
2000	0.216302	0.238545	0.925395

3000	0.232691	0.193954	0.935395
4000	0.207634	0.187870	0.936974
5000	0.190419	0.192051	0.939079
6000	0.191316	0.191797	0.940526
7000	0.179650	0.185396	0.942632
8000	0.136532	0.191887	0.944211
9000	0.125646	0.196465	0.947895
10000	0.131991	0.198724	0.944342
11000	0.134169	0.199233	0.943553
12000	0.128093	0.188066	0.946053
13000	0.134159	0.181824	0.948684
14000	0.124488	0.187128	0.948026
15000	0.129166	0.185813	0.948026

Writing model shards: 100% 1/1 [00:06<00:00, 6.64s/it]

Writing model shards: 100% 1/1 [00:06<00:00, 6.50s/it]

Writing model shards: 100% 1/1 [00:05<00:00, 5.54s/it]

Writing model shards: 100% 1/1 [00:05<00:00, 5.74s/it]

Writing model shards: 100% 1/1 [00:03<00:00, 3.18s/it]

Writing model shards: 100% 1/1 [00:06<00:00, 6.97s/it]

Writing model shards: 100% 1/1 [00:06<00:00, 6.18s/it]

Writing model shards: 100% 1/1 [00:05<00:00, 5.98s/it]

Writing model shards: 100% 1/1 [00:05<00:00, 5.67s/it]

Writing model shards: 100% 1/1 [00:05<00:00, 5.49s/

Step 9: Evaluate on Test Set

Step 9: Evaluate on Test Set

```
results = trainer.evaluate()
print(f"\nTest Accuracy: {results['eval_accuracy']:.4f}")
print(f"Test Loss: {results['eval_loss']:.4f}")
```



```
[475/475 00:07]

Test Accuracy: 0.9488
Test Loss: 0.1818
```

Step 10: Classify New Articles Using the Trained Model

The pipeline API handles tokenization, inference, and label mapping in one function call.

```
# Create classification pipeline from the trained model
classifier = pipeline("text-classification", model=model, tokenizer=

# Test on new unseen articles
test_articles = [
    "NASA successfully launches new Mars rover to search for signs o
    "Manchester United defeats Liverpool 3-1 in Premier League showd
    "Stock market surges as Federal Reserve signals interest rate cu
    "United Nations holds emergency session on escalating conflict i
    "Google announces breakthrough in quantum computing with new 100
    "India wins Cricket World Cup final against Australia in thrilli
]

print("Classification Results:")
print("=" * 70)
for article in test_articles:
    result = classifier(article)[0]
    print(f"\nArticle: {article}")
    print(f"Category: {result['label']} (Confidence: {result['score'
```

```
Classification Results:
======================================================================

Article: NASA successfully launches new Mars rover to search for sig
Category: Sci/Tech (Confidence: 99.65%)

Article: Manchester United defeats Liverpool 3-1 in Premier League s
Category: Sports (Confidence: 99.83%)

Article: Stock market surges as Federal Reserve signals interest rat
Category: Business (Confidence: 99.29%)

Article: United Nations holds emergency session on escalating confli
Category: World (Confidence: 99.88%)

Article: Google announces breakthrough in quantum computing with new
```

```
Category: Sci/Tech (Confidence: 98.19%)

Article: India wins Cricket World Cup final against Australia in thr
Category: World (Confidence: 98.83%)
```

Conclusion

We successfully fine-tuned DistilBERT on the AG News dataset using transfer learning through the Hugging Face library.

The model achieves approximately 94% accuracy on the test set after just 2 epochs of training, demonstrating the effectiveness of transfer learning for text classification tasks.

Key observations:

- Transfer learning allows us to leverage DistilBERT's pre-trained language understanding
- Fine-tuning requires minimal training time (approximately 15 minutes on a T4 GPU)
- The Hugging Face ecosystem simplifies the entire pipeline from data loading to model deployment